CMPT 225 FINAL PROJECT DOCUMENTATION

I used the same files from assignment #1 and copy and paste into the rubikscube folder. The big issue which was solving the cube was now finished. Now I need to figure out how to keep track of each move to solve the cube, then append it to a char array

Keeping all the same methods RubiksCube.java had, I can still parse the file, get rid of its leading and trailing white space to readline each position.

Once im able to figure it out how to append to the char[array] i will change that into a string, then Then filewrite into the output file.

Decided to use IDA* the iterative deepening algorithm. I think this makes more sense because unlike the BFs and DFS I don't need a static memory, I can just allocate more when I need to go deeper.

```
private static String solveIDAStar(RubiksCube start) {
    for (int depth = 0; depth <= MAX_DEPTH; depth++) {
        System.out.println("Searching depth: " + depth);
        String result = depthLimitedSearch(start, depth, "", -1);
        if (result != null) {
            return result;
        }
    }
    return null;
}
```

The method searches from depth 0 to MAX_DEPTH (global variable of int = 21), I decided to limit max depth to 21 based on a google search saying that most Rubik's Cubes can be solved within 20 moves.

```
private static String depthLimitedSearch(RubiksCube cube, int depth, String path, int lastMove) {
    if (depth == 0) {
        return cube.isSolved() ? path : null;
    }

    if (heuristic(cube) > depth) {
        return null;
    }

    for (int i = 0; i < MOVES.length; i++) {
        // No redundancy checking - allow all move sequences
```

```
        RubiksCube next = new RubiksCube(cube);
        next.applyMoves(String.valueOf(MOVES[i]));

        String result = depthLimitedSearch(next, depth - 1, path + MOVES[i], i);
        if (result != null) {
            return result;
        }
    }

    return null;
}
```

Originally in my IDA* implementation I had checked for redundancy, so I made a helper function and kept track of my last move, so if my last move was the same as the current I didn't allow it to happen. But I realized after looking at one of your solutions for the scrambles and rubikscube solver online that consecutive moves are needed, so I ended up ignoring easier solutions like the “FBBB” second solution.

Additionally my IDA* implementation was still extremely slow, so I searched up on youtube some implementations. Alot of the implementations added a heuristic value, which basically just counts all the wrong faces, then does (misplaced + 7) / 8 (rounding up) into RubiksCube.java so it cuts off the possibilities of checking impossible routes, for example searching for solution with max 3 moves, depth = 3, without heuristic value you have to explore all $6^3 = 216$ paths., With heuristic we check the value and f heuristic value is 5, (5 > 3), Immediately stop exploring this branch so you save exploring 216 - 1 = 215 paths.

```
public int getHeuristicValue() {
    int misplaced = 0;
    for (int face = 0; face < 6; face++) {
        char expectedColor = COLORS[face];
        for (int i = 0; i < SIZE; i++) {
            for (int j = 0; j < SIZE; j++) {
                if (cube[face][i][j] != expectedColor) {
                    misplaced++;
                }
            }
        }
    }
    return (misplaced + 7) / 8;
```

So what this on does is it counts the misplaced facelets on the array, then divide by 8 since there are 8 facelets.

Instead of making a character array then turning into a string then outputting onto the file I just decided to write directly to the output file as a giant string, the parser I made called writeSolution which is:

```
    private static void writeSolution(String filename, String solution) {
        try (PrintWriter writer = new PrintWriter(new
FileWriter(filename))) {
            writer.print(solution);
        } catch (IOException e) {
            System.err.println("Error writing solution: " +
e.getMessage());
        }
    }
```

Instead of making a character array and typecasting to a string, I implemented string cocantentation when the search method is called.

```
    private static String solveIDAStar(RubiksCube start) {
        for (int depth = 0; depth <= MAX_DEPTH; depth++) {
            System.out.println("depth: " + depth);
            String result = depthLimitedSearch(start, depth, "", -1);
            if (result != null) {
                return result;
            }
        }
        return null;
    }
```

(String result = depthLimitedSearch(next, depth - 1, path + MOVES[i], i);).

Once a solution is found, the string is written to the output file using PrintWriter, which outputs to a text file. The initial call is like in the pdf where it takes the two arguments, input then output:

```
 public static void main(String[] args) {
        if (args.length < 2) { (MUST HAVE AN INPUT AND OUTPUT FILE
SPECIFIED)
```

```
            System.out.println("usage: java rubikscube.Solver input_file
output_file");
            return;
        }

        try {
            RubiksCube cube = new RubiksCube(args[0]);
            String solution = solveIDAStar(cube); (CALL THE SEARCH METHOD)

            if (solution != null) {
                writeSolution(args[1], solution);
                System.out.println("Solution: " + solution);
            } else {
                writeSolution(args[1], "");
                System.out.println("No solution found");
            }

        } catch (Exception e) {
            System.err.println("Error: " + e.getMessage());
            e.printStackTrace();
        }
    }
```

11/23/2025

I decided to readd the redundancy method but make it better. So now some moves the RubiksCube can apply are double moves like FF but it cannot do it twice like (FF then FF) basically if it did it would go to the same place, but it would allow moves like FF then F because those are different moves, but not FF then F then F (same position) but FF, then F then FF will still work because its still in a different location.
None of my cases were working, i kept getting stuck at depth 14+. So I went to thse online rubiks cube solvers and see how they worked, looking at their inverse moves (F’ = FFF)
Was basically a 270 degree turn so I decided to also implement that so I dont have to do repetitive moves.

```java
private static void applyMove(RubiksCube cube, char move) {
    if (move == 'F') {
        cube.applyMoves("F");
    } else if (move == 'f') {
        cube.applyMoves("FF");
    } else if (move == 'L') {
        cube.applyMoves("L");
    } else if (move == 'l') {
        cube.applyMoves("LL");
    } else if (move == 'R') {
        cube.applyMoves("R");
    } else if (move == 'r') {
        cube.applyMoves("RR");
    } else if (move == 'U') {
        cube.applyMoves("U");
    } else if (move == 'u') {
        cube.applyMoves("UU");
    } else if (move == 'D') {
        cube.applyMoves("D");
    } else if (move == 'd') {
        cube.applyMoves("DD");
    } else if (move == 'B') {
        cube.applyMoves("B");
    } else if (move == 'b') {
        cube.applyMoves("BB");
    }
}
```

11/24/2025

Nvm i decided to change the movesets to

```
public class Solver {
    // All moves including inverses
    private static final String[] MOVES = {"F", "L", "R", "U", "D", "B",
                                           "FF", "LL", "RR", "UU", "DD",
"BB",
                                           "FFF", "LLL", "RRR", "UUU",
"DDD", "BBB"};
    private static final int MAX_DEPTH = 21;
```

But inside my depth limiteds search I added:

```
        for (int i = 0; i < MOVES.length && solution == null; i++) {
            String move = MOVES[i];
            char moveFace = move.charAt(0);

            if (lastMove >= 0) {
                String lastMoveStr = MOVES[lastMove];
                char lastMoveFace = lastMoveStr.charAt(0);
                if (moveFace == lastMoveFace) {
                    continue;
                }
            }

            if (lastMove >= 0 && path.length() > 0) {
                String lastMoveStr = MOVES[lastMove];

                if (areInverseMoves(lastMoveStr, move)) {
                    continue;
                }

                if ((lastMoveStr.length() == 1 && move.length() == 2 &&
lastMoveStr.charAt(0) == moveFace) ||
                    (lastMoveStr.length() == 2 && move.length() == 1 &&
lastMoveStr.charAt(0) == moveFace)) {
                    continue;
                }
            }
```

So if previous moves was also F ill skip any variation of F i have like F, FF, and FFF
I brought back the redundancy and just added more moves because when I was looking at the first scramble solution online, they used inverses of U' which is really just UUU.

11/25/2025
Looking at scramble 2 I get solution:
After FBBB:
  RRR
  OOO
  OOO
BGGWWWBBGYYY
BGGWWWBBGYYY
BGGWWWBBGYYY
  RRR
  RRR
  OOO
So looks like my moveset is a bit different but when i apply BF:
  OOO
  OOO
  OOO
GGGWWWBBBYYY
GGGWWWBBBYYY
GGGWWWBBBYYY
  RRR
  RRR
  RRR

It matches the solved state.

Added solution tracking:

```
    private static String solution = null;
```

Which:

```
    private static String solveIDAStar(RubiksCube start) {
        solution = null;
        int startDepth = heuristic(start);

        for (int depth = startDepth; depth <= MAX_DEPTH && solution ==
null; depth++) {
            System.out.println("depth: " + depth);
            depthLimitedSearch(start, depth, "", -1);
            if (solution != null) {
                return solution;
            }
        }
        return null;
    }
```

```
    public static void main(String[] args) {
        if (args.length < 2) {
            System.out.println("not specified");
            System.out.println("usage: java rubikscube.Solver input_file
output_file");
            return;
        }

        try {
            RubiksCube cube = new RubiksCube(args[0]);

            String solution = solveIDAStar(cube);

            if (solution != null) {
                solution = optimizeSolution(solution);
                writeSolution(args[1], solution);
                System.out.println("Solution found: " + solution);
                System.out.println("Length: " + solution.length() + "
moves");
            } else {
                writeSolution(args[1], "");
                System.out.println("No solution found within depth " +
MAX_DEPTH);
            }

        } catch (Exception e) {
            System.err.println("Error: " + e.getMessage());
            e.printStackTrace();
        }
    }
```

In my main i added a solution output so that when i run it in command prompt it actually shows me the string instead of having ot open up the .txt.

I changed the search method a bit:

```
    private static String solveIDAStar(RubiksCube start) {
        solution = null;
        int startDepth = heuristic(start);

        for (int depth = startDepth; depth <= MAX_DEPTH && solution ==
null; depth++) {
```

```
            System.out.println("depth: " + depth);
            depthLimitedSearch(start, depth, "", -1);
            if (solution != null) {
                return solution;
            }
        }
        return null;
    }
```

In the for loop it avoids exploring depths that the heuristic when called is too great. The IDA* itself:

```
    private static void depthLimitedSearch(RubiksCube cube, int depth, String path, int lastMove) {
        if (solution != null) return;

        if (cube.isSolved()) {
            solution = path;
            return;
        }

        if (depth == 0) return;

        int h = heuristic(cube);
        if (h > depth) {
            return;
        }

        for (int i = 0; i < MOVES.length && solution == null; i++) {
            String move = MOVES[i];
            char moveFace = move.charAt(0);

            if (lastMove >= 0) {
                String lastMoveStr = MOVES[lastMove];
                char lastMoveFace = lastMoveStr.charAt(0);
                if (moveFace == lastMoveFace) {
                    continue;
                }
            }

            if (lastMove >= 0 && path.length() > 0) {
                String lastMoveStr = MOVES[lastMove];
```

```
                if (areInverseMoves(lastMoveStr, move)) {
                    continue;
                }

                if ((lastMoveStr.length() == 1 && move.length() == 2 &&
lastMoveStr.charAt(0) == moveFace) ||
                    (lastMoveStr.length() == 2 && move.length() == 1 &&
lastMoveStr.charAt(0) == moveFace)) {
                    continue;
                }
            }

            RubiksCube next = new RubiksCube(cube);
            next.applyMoves(move);

            depthLimitedSearch(next, depth - 1, path + move, i);
        }
    }
```

Takes a recursive call and will break out of recursion if:

```
        if (solution != null) return;
```

```
        if (cube.isSolved()) {
            solution = path;
            return;
        }

        if (depth == 0) return;

        int h = heuristic(cube);
        if (h > depth) {
            return;
        }
```

So first if its been solved elsewhere it will stop searching
Check if itws solved, if it is, set solution = to path and return (so now the solution has the string of moves we took to solve)
And if we reached the end depth just return (nowhere else to go)
Below the if statements:

```
        int h = heuristic(cube);
        if (h > depth) {
            return;
        }
```

So if the heuristic says estimated moves is greater than depth, just give up searching this branch because it will take too long
Here is the redundant move checker i readded:

```
            if (lastMove >= 0) {
                String lastMoveStr = MOVES[lastMove];
                char lastMoveFace = lastMoveStr.charAt(0);
                if (moveFace == lastMoveFace) {
                    continue;
                }
            }
```

Additionally:

```
                if (areInverseMoves(lastMoveStr, move)) {
                    continue;
                }
```

This line helps me check if i've done F, FFF (basically led to nowhere) or FF then FF. So it prevents me from doing moves that just place me back where i started

```
            RubiksCube next = new RubiksCube(cube);
            next.applyMoves(move);

            depthLimitedSearch(next, depth - 1, path + move, i);
        }
```

This is for the recursive calls, we copy the cube instead of then apply chosen move and continue to search at depth-1 then record the path string and update

From earlier this is where the function is called from:

```
    private static boolean areInverseMoves(String move1, String move2) {
        if (move1.charAt(0) != move2.charAt(0)) return false;

        int len1 = move1.length();
        int len2 = move2.length();
```

```
        return (len1 == 1 && len2 == 3) ||
               (len1 == 3 && len2 == 1) ||
               (len1 == 2 && len2 == 2);
    }
```

So bvasically checks X + XXX, XXX + X and XX + XX
11/26/2025
https://www.reddit.com/r/algorithms/comments/wgfloz/heuristic_function_for_a_rubiks_cube/
https://en.wikipedia.org/wiki/Optimal_solutions_for_the_Rubik%27s_Cube
Heuristic

```
    private static int cornerManhattan(RubiksCube cube) {
        int totalDistance = 0;

        int[][][] corners = {
            {{0,0,0}, {1,0,2}, {4,0,0}}, // ULB
            {{0,0,2}, {3,0,0}, {4,0,2}}, // URB
            {{0,2,0}, {1,2,2}, {2,0,0}}, // ULF
            {{0,2,2}, {2,0,2}, {3,0,2}}, // URF
            {{5,0,0}, {1,0,0}, {2,2,0}}, // DLF
            {{5,0,2}, {2,2,2}, {3,2,0}}, // DRF
            {{5,2,0}, {1,2,0}, {4,2,2}}, // DLB
            {{5,2,2}, {3,2,2}, {4,2,0}}  // DRB
        };

        for (int c = 0; c < 8; c++) {
            int[][] corner = corners[c];

            char color1 = cube.getFacelet(corner[0][0], corner[0][1],
corner[0][2]);
            char color2 = cube.getFacelet(corner[1][0], corner[1][1],
corner[1][2]);
            char color3 = cube.getFacelet(corner[2][0], corner[2][1],
corner[2][2]);

            int minDistance = 3;

            for (int targetCorner = 0; targetCorner < 8; targetCorner++) {
                int[][] target = corners[targetCorner];
                char targetColor1 = RubiksCube.getColor(target[0][0]);
                char targetColor2 = RubiksCube.getColor(target[1][0]);
```

```
                char targetColor3 = RubiksCube.getColor(target[2][0]);

                boolean matches = (color1 == targetColor1 || color1 ==
targetColor2 || color1 == targetColor3) &&
                                (color2 == targetColor1 || color2 ==
targetColor2 || color2 == targetColor3) &&
                                (color3 == targetColor1 || color3 ==
targetColor2 || color3 == targetColor3);

                if (matches) {
                    int distance = (c == targetCorner) ? 0 : 1;
                    minDistance = Math.min(minDistance, distance);
                }
            }

            totalDistance += minDistance;
        }

        return (totalDistance + 3) / 4;
    }

    private static int edgeManhattan(RubiksCube cube) {
        int totalDistance = 0;

        int[][][] edges = {
            {{0,0,1}, {4,0,1}}, // UB
            {{0,1,0}, {1,1,2}}, // UL
            {{0,1,2}, {3,1,0}}, // UR
            {{0,2,1}, {2,0,1}}, // UF
            {{5,0,1}, {2,2,1}}, // DF
            {{5,1,0}, {1,1,0}}, // DL
            {{5,1,2}, {3,1,2}}, // DR
            {{5,2,1}, {4,2,1}}, // DB
            {{1,0,1}, {4,1,2}}, // LB
            {{1,2,1}, {2,1,0}}, // LF
            {{3,0,1}, {4,1,0}}, // RB
            {{3,2,1}, {2,1,2}}  // RF
        };

        for (int e = 0; e < 12; e++) {
```

```
            int[][] edge = edges[e];

            char color1 = cube.getFacelet(edge[0][0], edge[0][1],
edge[0][2]);
            char color2 = cube.getFacelet(edge[1][0], edge[1][1],
edge[1][2]);

            int minDistance = 2;

            for (int targetEdge = 0; targetEdge < 12; targetEdge++) {
                int[][] target = edges[targetEdge];
                char targetColor1 = RubiksCube.getColor(target[0][0]);
                char targetColor2 = RubiksCube.getColor(target[1][0]);

                boolean matches = (color1 == targetColor1 || color1 ==
targetColor2) &&
                                  (color2 == targetColor1 || color2 ==
targetColor2);

                if (matches) {
                    int distance = (e == targetEdge) ? 0 : 1;
                    minDistance = Math.min(minDistance, distance);
                }
            }

            totalDistance += minDistance;
        }

        return (totalDistance + 2) / 3;
    }
```

```java
    private static int cornerManhattan(RubiksCube cube) {
        int totalDistance = 0;

        int[][][] corners = {
            {{0,0,0}, {1,0,2}, {4,0,0}}, // ULB
            {{0,0,2}, {3,0,0}, {4,0,2}}, // URB
            {{0,2,0}, {1,2,2}, {2,0,0}}, // ULF
            {{0,2,2}, {2,0,2}, {3,0,2}}, // URF
            {{5,0,0}, {1,0,0}, {2,2,0}}, // DLF
            {{5,0,2}, {2,2,2}, {3,2,0}}, // DRF
            {{5,2,0}, {1,2,0}, {4,2,2}}, // DLB
            {{5,2,2}, {3,2,2}, {4,2,0}}  // DRB
        };
```

This function looks at all 8 corner cubies on the cube. Each corner cube has 3 stickers. Each {{face,row,col}, {face,row,col}, {face,row,col}} triple tells us where the 3 stickers of that corner are on the cube. Then try to figure out which corner it belongs to. Then loop over all 8 corners, for each target corner, we look up what its 3 colors should be in the solved cube using RubiksCube.getColor(faceIndex). If it is already in that correct corner (c == targetCorner) then the distance is 0. If it's in the wrong corner then we say the distance is at least 1 (so just put 1) then add to total distance. Then at the end return (totalDistance + 3) / 4; (divide by 4 because if 8 corners are wrong, in the best case you could fix them in about 8 / 4 = 2 moves.

```java
    private static int edgeManhattan(RubiksCube cube) {
        int totalDistance = 0;

        int[][][] edges = {
            {{0,0,1}, {4,0,1}}, // UB
            {{0,1,0}, {1,1,2}}, // UL
            {{0,1,2}, {3,1,0}}, // UR
            {{0,2,1}, {2,0,1}}, // UF
            {{5,0,1}, {2,2,1}}, // DF
            {{5,1,0}, {1,1,0}}, // DL
            {{5,1,2}, {3,1,2}}, // DR
            {{5,2,1}, {4,2,1}}, // DB
            {{1,0,1}, {4,1,2}}, // LB
            {{1,2,1}, {2,1,0}}, // LF
            {{3,0,1}, {4,1,0}}, // RB
            {{3,2,1}, {2,1,2}}  // RF
        };
```

```
        for (int e = 0; e < 12; e++) {
            int[][] edge = edges[e];

            char color1 = cube.getFacelet(edge[0][0], edge[0][1],
edge[0][2]);
            char color2 = cube.getFacelet(edge[1][0], edge[1][1],
edge[1][2]);

            int minDistance = 2;

            for (int targetEdge = 0; targetEdge < 12; targetEdge++) {
                int[][] target = edges[targetEdge];
                char targetColor1 = RubiksCube.getColor(target[0][0]);
                char targetColor2 = RubiksCube.getColor(target[1][0]);

                boolean matches = (color1 == targetColor1 || color1 ==
targetColor2) &&
                                  (color2 == targetColor1 || color2 ==
targetColor2);

                if (matches) {
                    int distance = (e == targetEdge) ? 0 : 1;
                    minDistance = Math.min(minDistance, distance);
                }
            }

            totalDistance += minDistance;
        }

        return (totalDistance + 2) / 3;
    }
```

The edge heuristic is similar, like how we list all out all the possible edges in an array. We still loop over the 12 edges on the cube and read its two colours. Then like the previous corner manhattan.

Additionally for main i added a length

```
                System.out.println("Length: " + solution.length() + "
moves");
```

But its a little bit inaccurate that it just counts the letters even though one move could have been FF or FFF. I was thinking i could just put a variable called moves and increment once everytime

it did a move but then i'd have to also make it say moves–; if i backtracked or anything so i just decided not to add it and stick with a semi working length.
Again for the edge heuristic the: `return (totalDistance + 2) / 3;`
Gives us the lower bounds based on how many of the 12 edge pieces it counted wrong. (again each wrong one is at least distance 1, correct pieces are distance 0.
I also added "optimized solutions" Because like earlier FFF then F is basically a full 360. This method looks at the output string of moves and changes the repeated moves on the same face into simpler forms, so the most you'd see is a 270 degree turn like FFF. If the solution length is 0 or 1, return. else, read the first move and keep incrementing counter.

```
if (solution.length() <= 1) return solution;
```

```
StringBuilder optimized = new StringBuilder();
char currentFace = solution.charAt(0);
int count = 1;
```

Loop through the rest of the moves and if the next character is the same as the last, increase the counter. If it is different, then separate the block of repeated moves using getOptimizedMove and reset counting. After the loop ends, check the final block of repeats for more than 3 of the same moves, then return the new string. To clarify, my IDA* search uses the set moves like("F", "FF", "FFF") and is pruned inside depthLimitedSearch (no same face twice in a row, no immediate inverses). That pruning stops redundant sequences then optimizeSolution only runs after a solution has already been found. Looking at the string of moves, groups consecutive face turns and reduces them into the shortest equivalent ("", "F", "FF", or "FFF"). So it doesn't help the search go faster; it just makes the final printed solution cleaner and shorter.

11/30/2025

Removed optimized, because during my depthsearch i already ignore redundant moves so theres no point in calling optimize solution if we never explore redundant possibilities in the first place.
11/31/2025

Attempted to implement BFS instead of IDA* but got stuck on a couple more scrambles like scramble 4.

12/1/2025